Address		Name
Mem.	I/O	
$20	$00	TWBR
$21	$01	TWSR
$22	$02	TWAR
$23	$03	TWDR
$24	$04	ADCL
$25	$05	ADCH
$26	$06	ADCSRA
$27	$07	ADMUX
$28	$08	ACSR
$29	$09	UBRRL
$2A	$0A	UCSRB
$2B	$0B	UCSRA
$2C	$0C	UDR
$2D	$0D	SPCR
$2E	$0E	SPSR
$2F	$0F	SPDR
$30	$10	PIND
$31	$11	DDRD
$32	$12	PORTD
$33	$13	PINC
$34	$14	DDRC
$35	$15	PORTC

Address		Name
Mem.	I/O	
$36	$16	PINB
$37	$17	DDRB
$38	$18	PORTB
$39	$19	PINA
$3A	$1A	DDRA
$3B	$1B	PORTA
$3C	$1C	EECR
$3D	$1D	EEDR
$3E	$1E	EEARL
$3F	$1F	EEARH
$40	$20	UBRRC
		UBRRH
$41	$21	WDTCR
$42	$22	ASSR
$43	$23	OCR2
$44	$24	TCNT2
$45	$25	TCCR2
$46	$26	ICR1L
$47	$27	ICR1H
$48	$28	OCR1BL
$49	$29	OCR1BH
$4A	$2A	OCR1AL

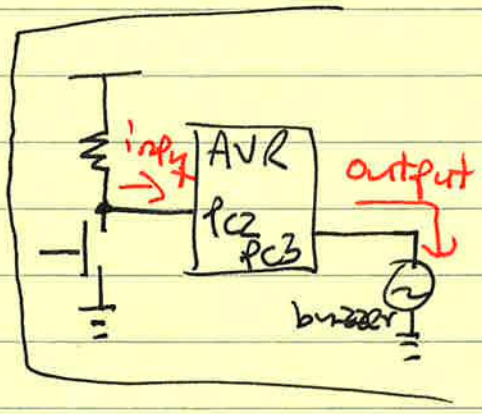
Address		Name
Mem.	I/O	
$4B	$2B	OCR1AH
$4C	$2C	TCNT1L
$4D	$2D	TCNT1H
$4E	$2E	TCCR1B
$4F	$2F	TCCR1A
$50	$30	SFIOR
$51	$31	OCDR
		OSCCAL
$52	$32	TCNT0
$53	$33	TCCR0
$54	$34	MCUCSR
$55	$35	MCUCR
$56	$36	TWCR
$57	$37	SPMCR
$58	$38	TIFR
$59	$39	TIMSK
$5A	$3A	GIFR
$5B	$3B	GICR
$5C	$3C	OCR0
$5D	$3D	SPL
$5E	$3E	SPH
$5F	$3F	SREG

Note: Although memory address $20-$5F is set aside for I /O registers (SFR) we can access them as I /O locaticns with addresses starting at $00.

Figure 2-7. I/O Registers of the ATmega32 and Their Data Memory Address Locations

Assume that bit PC2 is an input and represents the condition of a door alarm.

If it goes HIGH, then the door is open. Monitor the bit continuously. Whenever it goes HIGH, send a high-to-low signal to PC3 for a buzzer for 30 iterations.

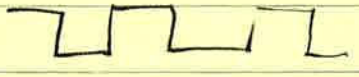



```
.ORG 0
CBI  DDRC,2 ; PC2 => input (0)
SBI  DDRC,3 ; PC3 => output (1)
SBI  PORTC,2 ; PC2 => 1 => enable pull-up resistor

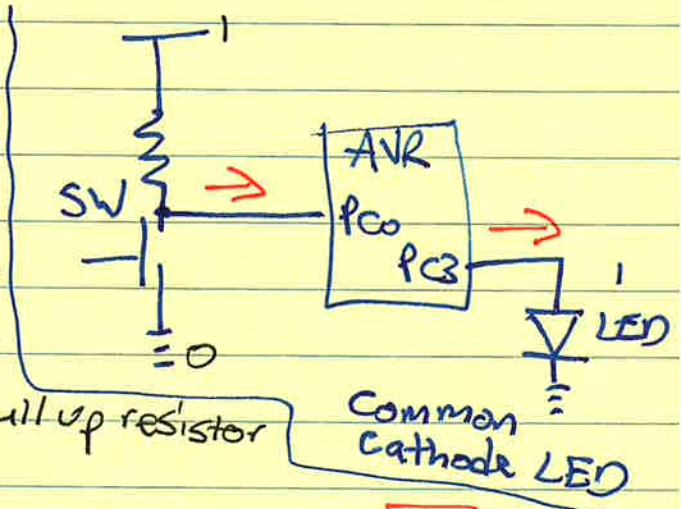
Here: SBIS PINC,2 ; check to see if door is open
      RJMP Here   ; PC2=1 continuously
      LDI  R16,30 ; PC2=0 => door open
loop: SBI  PORTC,3 ;
      Call Delay
      CBI  PORTC,3 ;
      Call Delay
      DEC R16  ; dec loop count
      BRNE loop
      RJMP Here ; back to monitoring the
                  door condition
```



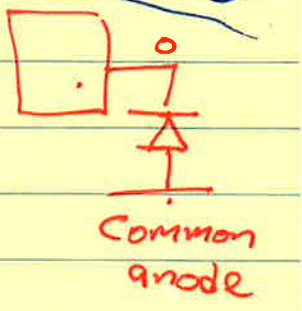




A switch (SW) is connected to pin PC0 and an LED to pin PC3. Write a program to get the status of SW and send it to the LED.




```
.ORG 0
CBI DDRC,0 ; PC0 input
SBI DDRC,3 ; PC3 output
SBI PORTC,0 ; pull up resistor
```




```
AGAIN: SBIC PINC,0 ; SW=0?
       CALL LEDON  ; SW=1
       SBIS PINC,0 ; SW=1?
       CALL LEDOFF
       RJMP AGAIN
```

```
.ORG 0x30
LEDON: SBI PORTC,3
       RET
.ORG 0x40
LEDOFF: CBI PORTC,3
        RET
```

Assume that data memory locations $160-$163 have hex data. Write a program to access (add) these values and place the result in locations $170 (LSB) and $171 (MSB)

```
.EQU Low_Byte = 0x170
.EQU High_Byte=0x171

.ORG 0
LDI R17,4 ; # bytes to add
LDI R18,0 ;
LDI R19,0 ; initialize 16bit sum R19R18
LDI XL,0x60
LDI XH,0x01 ; X=0x0160
Next_Num: LD R20,X+ ; get data pointed to by
                    ; X -> copy into R20
ADD R18,R20 ; add to low byte sum
BRCC No_Carry ; carry?
INC R19 ; yes -> inc high byte of sum

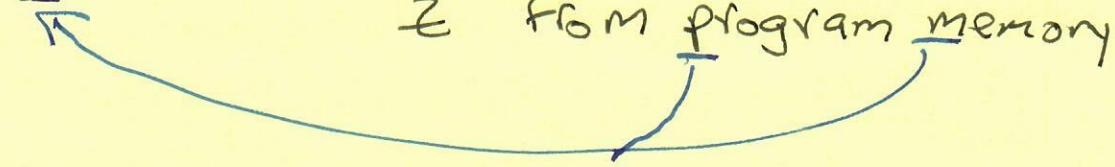
No_Carry: DEC R17 ; dec # numbers to add
BRNE Next_Num ; do for all # numbers to add
ST Low_Byte,R18 ; store result
ST High_Byte,R19
```

LD R20,X-
LD R20,X+
LD R20,-X
LDS

Program memory Indirect Addressing

LD ⇄ data

LPM R16, Z ; load value pointed to by Z from program memory



LPM R16, Z+

- look up tables

16 bit Code Memory

Low	High	Address
0000 0000 0000 0000	0000 0000 0000 0001	0000 0000 0000 0000
0000 0000 0000 0010	0000 0000 0000 0011	0000 0000 0000 0001
0000 0000 0000 0100	0000 0000 0000 0101	0000 0000 0000 0010
0000 0000 0000 0110	0000 0000 0000 0111	0000 0000 0000 0011
0000 0000 0000 1000	0000 0000 0000 1001	0000 0000 0000 0100
0000 0000 0000 1010	0000 0000 0000 1011	0000 0000 0000 0101
⋮	⋮	
1111 1111 1111 1100	1111 1111 1111 1101	0111 1111 1111 1110
1111 1111 1111 1110	1111 1111 1111 1111	0111 1111 1111 1111

Figure 6-13a. Values of Z (in Binary)

low 8bits, high 8bits, Code memory location

Low	High	Address
$0000	$0001	$0000
$0002	$0003	$0001
$0004	$0005	$0002
$0006	$0007	$0003
$0008	$0009	$0004
$000A	$000B	$0005
$600 'U'	$601, 'p', 60c 'c'	$300
$FFFC	$FFFD	$7FFE
$FFFE	$FFFF	$7FFF

Figure 6-13b. Values of Z

Assume that program ROM space starting at 0x300 containing the string "Upcoming Test". Write a program to send all the characters to PORTC one byte at a time.

```
.ORG 0
LDI R16, 0xFF
OUT DDRC, R16; PORTC output port
LDI R17, 13; # characters to send to PORTC
LDI ZL, LOW(Message<<1);
LDI ZH, HIGH(Message<<1);
NOTDONE: LPM R16, Z+ ; load current character, increment Z
OUT PORTC, R16 ; output current character to PORTC
DEC R17 ; decrement character count
BRNE NOTDONE ; get all characters output
                to PORTC

0x600
.ORG 0x300
Message: .DB "Upcoming Test"
```

Message location
0011 0000 0000
Z = 0110 0000 0000
0x6 0 0

Addition & Subtraction

ADD ADC
SUB SBC SUBI Rd, k

Ex) 2762h
- 1296h

R17 R16
R19 R18

```
LDI R16, 0x62
LDI R17, 0x27
LDI R18, 0x96
LDI R19, 0x12
SUB R16, R18 ; low byte subtraction
SBC R17, R19 ; high byte subtraction
```

16 bit difference
is 14CCh
R17 = 14h
R16 = CCh

R17 = R17 - R19 - C

62
-96h

27h R17
-12h -R19 => 12h => 0001 0010

2s comp 1110 1101
+ 1
1110 1110

11 1 11
0010 0111
+1110 1110
---
1 0001 0101 (15h)
^
C*
invert C* = 0

<=

1001 0110
0110 1001
+ 1
0110 1010

62h 0110 0010
+(-96h) +0110 1010
=> 0 1100 1000
C
invert
C=1 borrow

R17-R19= 00010101

Actual C is
C* OR C** = 0

Invert C** = 0

R17
=> -R19
-C

=> C = 1 from low byte SUB

so 0000 0001 => 2s comp
1111 1110
+ 1
1111 1111

15h 0001 0101
-1 +1111 1111

C**=>1 0001 0100

<=

R17-R19-C = 0001 0100

R16-R18 = 1100 1100

Multiplication

SREG
Z
C

MUL Rd, Rr ; Rd, Rr unsigned
R1 high byte of product
R0 low byte of product

SP = 0x0000
move SP
to avoid
contaminating
R0

High(RAMEND) → SPH
Low(RAMEND) → SPL

0x38
x 0x86
0x1D50
R1 R0

```
LDI R17, 0x38
LDI R18, 0x86
MUL R17, R18
```

SREG MULS Rd, Rr ; Rd x Rr signed
R1 high byte of product
R0 low byte of product

Z C Only
No signed number
flags affected

```
  0011   (+3)
x 1000   (-8)
------
  00000  initial sum
 00000
------
 000000
 00000
------
 0000000
 00000
------
 1110100
```

```
 0011
(2s comp
 1100
+   1
------
 1101
```

Division

- AVR has no division instruction
- perform division based on repeated subtractions

$$\begin{array}{r} 7 \text{ R } 5 \\ 10 \overline{)\,75} \end{array}$$

$*< 100$

```
75
1 -10
 65
2 -10
 55
3 -10
 45
4 -10
 35
5 -10
 25
6 -10
 15
7 -10
  5
8 -10
 -5
8-1=7
```

```
.DEF NUM=R20
.DEF DENOM=R21
.DEF QUOT=R22

.ORG
LDI  NUM, 75
LDI  DENOM, 10
CLR  QUOT; QUOT=0
loop: INC QUOT
SUB  NUM, DENOM
BRCC loop; if C=0
                no borrow so continue
DEC  QUOT; went 1 iteration too far
ADD  NUM, DENOM; get remainder in NUM
```